

Scalable Real-time Detection of AI-Generated Arabic Text:

A Distributed Data Pipeline Approach

MSBDA-801 Big Data Analytics

Final Course Graduation Team Project

Abstract

Generative AI has grown at a rapid pace, generating machine-produced text in various languages, including Arabic. This project aims to detect AI-generated Arabic abstracts with a scalable distributed pipeline via Big Data technologies and machine learning techniques. The system was developed by using Apache Spark and PySpark in a distributed processing environment. The data for this research were the KFUPM-JRCAI Arabic Generated Abstracts data from Hugging Face.

The pipeline proposed consists of distributed data acquisition, Arabic-specific preprocessing, scalable feature engineering, machine learning model training, real-time simulation of the pipeline stream, and performance benchmarking.

The traditional stylometric features and TF-IDF representations were joined to enhance the classification performance. The three machine learning models that were tested were: Logistic Regression, Random Forest, and Support Vector Machine (SVM). The performance of the SVM model on the experimental results showed that it has the highest overall accuracy of about 97.3%, F1 score of 97.3%, and ROC-AUC of 99.1%. Moreover, Spark Structured Streaming was used to simulate realtime prediction of Arabic text streams. The project showcases the

power of a distributed machine learning pipeline in enabling scalable, AI-driven Arabic text detection.

1. Introduction

Large language models and generative AI systems have greatly boosted the production of synthetic text content. These technologies have many benefits, but also come with some problems like misinformation, plagiarism, academic integrity, and content generation. The detection of AI-generated text has thus emerged as a critical research challenge. The Arabic Natural Language Processing (NLP) also comes with its own set of complexities, such as Arabic morphology, diacritics, variations in characters, and the diversity of writing styles. The Arabic text preprocessing and feature extraction are challenging in traditional NLP systems. Thus, distributed systems with scalability are required to process large Arabic datasets effectively. This project involves the development of a distributed pipeline that can be scaled to detect if Arabic abstracts are written by human(s) or AI models. This project includes distributed preprocessing, scalable feature engineering, machine learning classification and real-time stream processing with Apache Spark (Devlin et al., 2019).

1.1 Project Objectives

Primary goals of this project are:

- Construct a distributed Arabic text processing pipeline with Apache Spark.
- Apply Arabic-specific preprocessing techniques.
- Apply scalable feature engineering techniques.
- Train and test various machine learning models.
- Deploy the best-performing model in a streaming environment.
- Benchmark batch and stream processing performance.
- Analyze model behavior and feature importance.

2. Related Work

With the rise of generative language models like GPT, LLaMA, and JAIS, AI-driven text detection has emerged as a rapidly expanding field of research (Devlin et al., 2019). Stylometric analysis, statistical analysis, transformer embedding and deep learning architectures have been used in previous studies for AI text classification (Jurafsky & Martin, 2023). Additionally, popular distributed Big Data frameworks like Apache Spark are well suited to scalable NLP applications due to their capabilities in parallel processing, distributed machine learning, and streaming applications (Salloum et al., 2016). Efficient implementations of machine learning algorithms for high-dimensional text data are available in Spark's MLlib. Traditional feature engineering methods like TF-IDF, stylometric patterns, punctuation frequency, and sentence structure continue to be effective at differentiating between human-written and AI-generated content (Manning, 2008). Further, distributed streaming architectures like Lambda Architecture and Kappa Architecture are widely used for scalable real-time analytics. Lambda Architecture uses both batch processing and stream processing layers, and Kappa Architecture is more about stream processing pipelines. These architectures enable low-latency analytics, fault tolerance, and scalability in a Big Data system (Dean & Ghemawat, 2008).

In this project, distributed NLP preprocessing, stylometric features, TF-IDF representation, and Spark Structured Streaming are integrated in a single scalable pipeline for AI detection of Arabic text.

3. Dataset Description

3.1 Dataset Source

The dataset used in this project is:

KFUPM-JRCAI/arabic-generated-abstracts

Source:

<https://huggingface.co/datasets/KFUPM-JRCAI/arabic-generated-abstracts>

The dataset contains Arabic research abstracts generated using different AI models and human-written originals (KFUPM-JRCAI Dataset, 2024).

3.2 Dataset Structure

Sample Dataset Examples:

- Human-written Arabic research abstracts
- AI-generated abstracts from OpenAI models
- AI-generated abstracts from JAIS models
- AI-generated abstracts from LLaMA models
- AI-generated abstracts from ALLAM models

The dataset consists of three generation subsets:

Generation Method	Samples Description	
by_polishing	2,851	AI-polished human abstracts
from_title	2,963	Generation using titles only
from_title_and_content	2,574	Generation using title and content
Total	8,388	Combined dataset

Each sample contains:

- original_abstract
- allam_generated_abstract
- jais_generated_abstract
- llama_generated_abstract
- openai_generated_abstract

3.3 Final Dataset Construction

Human-written abstracts were labeled as class 0, while AI-generated abstracts were labeled as class 1. All of the subsets were then combined and the final data set contained about 41,940 text samples.

3.4 Label Distribution

Label Meaning		Count
0	Human-written	8,388
1	AI-generated	33,552

4. Methodology

4.1 Big Data Environment Setup

The project was implemented using:

- Google Colab
- PySpark
- Apache Spark MLlib
- Python
- Spark Structured Streaming

In Colab, Spark sessions were used to set up a distributed processing environment (Apache Spark Documentation, 2024). The dataset was downloaded directly using the Hugging Face datasets library.

4.2 Data Acquisition and Storage

The data was loaded from the Hugging Face datasets API and transformed into a Pandas DataFrame prior to distributed processing in Spark.

The processed data was stored using:

- CSV format
- Parquet format

Processed data was persisted using Apache Parquet, a compressed columnar format, to optimize I/O operations and ensure distributed scalability (Dean & Ghemawat, 2008).

4.3 Arabic Text Preprocessing

Arabic-specific preprocessing performed using Spark UDFs

The preprocessing pipeline included:

- Arabic normalization
- Diacritics removal
- Removal of special characters
- Stopword removal
- Basic stemming

Examples of preprocessing operations:

- Converting Arabic character variants into normalized forms.
- Removing punctuation and numeric values.
- Filtering common Arabic stopwords.

The processed text was stored in a distributed Spark DataFrame (Jurafsky & Martin, 2023).

4.4 Exploratory Data Analysis (EDA)

```
spark_df.groupBy("label").count().show()
```

```
+-----+-----+
|label|count|
+-----+-----+
|    1|33552|
|    0| 8388|
+-----+-----+
```

Figure 1: Dataset label distribution visualization.

```
+-----+-----+
|word|count|
+-----+-----+
| 99134|علي|
| 75976|الي|
|49607|البح|
|48972|الدر|
|39865|الاس|
| 38408|ان|
|33133|التي|
|30468|الجز|
|29978|المت|
| 29650|بين|
+-----+-----+
only showing top 10 rows
```

Figure 2: Word frequency distribution from distributed Spark analysis.

Figure 3: Bigram/N-gram frequency analysis.

Figure 4: Vocabulary richness and TTR analysis results.

The following analyses were performed:

- Word frequency distribution
- N-gram analysis
- Vocabulary richness (Type-Token Ratio)
- Corpus statistics using MapReduce simulation

4.4.1 Type-Token Ratio (TTR)

The calculated TTR value was approximately:

$$\text{TTR} = 0.00587$$

This indicates a large vocabulary size across the Arabic corpus.

4.5 Feature Engineering

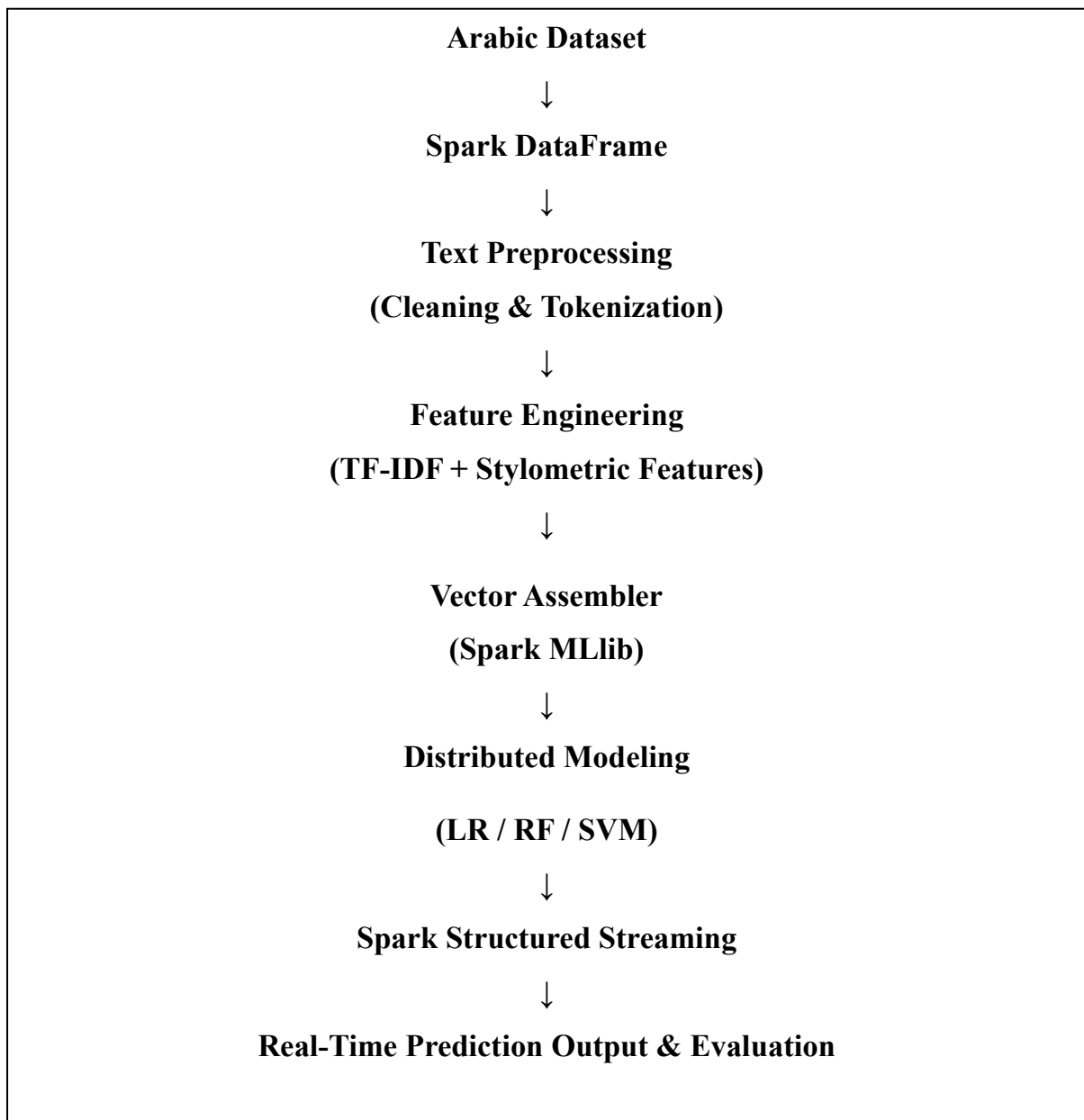


Figure 5: Distributed feature engineering pipeline architecture.

```

... +-----+
|           features | label |
+-----+
| (10005, [36, 64, 218... | 0 |
| (10005, [36, 218, 34... | 1 |
| (10005, [83, 249, 50... | 1 |
| (10005, [218, 500, 8... | 1 |
| (10005, [218, 249, 2... | 1 |
+-----+
only showing top 5 rows

```

Figure 6: TF-IDF feature vector generation process.

Combined scalable TF-IDF representation with stylometric features in feature engineering (Salton & Buckley, 1988).

4.5.1 Assigned Stylometric Features

The following assigned features were implemented:

Feature ID Description	
6	Multiple elongations
29	Semicolon frequency
52	Arabic interjections
75	Active voice approximation
98	Simplified BERT-style representation

4.5.2 TF-IDF Representation

TF-IDF vectorization was implemented using Spark MLlib.

Pipeline stages included:

- Tokenization
- Stopword filtering
- HashingTF
- Inverse Document Frequency (IDF)

A feature vector of 10,000 dimensions was generated and combined with the five stylometric features (Pedregosa et al., 2011).

4.6 Distributed Modeling

Three machine learning models were trained using Spark MLlib, which encapsulated Tokenization, HashingTF, IDF, and Vector Assembler to prevent Data Leakage during the 80/20 train-test split:

- Logistic Regression
- Random Forest
- Linear Support Vector Machine (SVM)

The dataset was split into:

- Training set: 70%
- Validation set: 15%
- Test set: 15%

4.7 Stream Processing

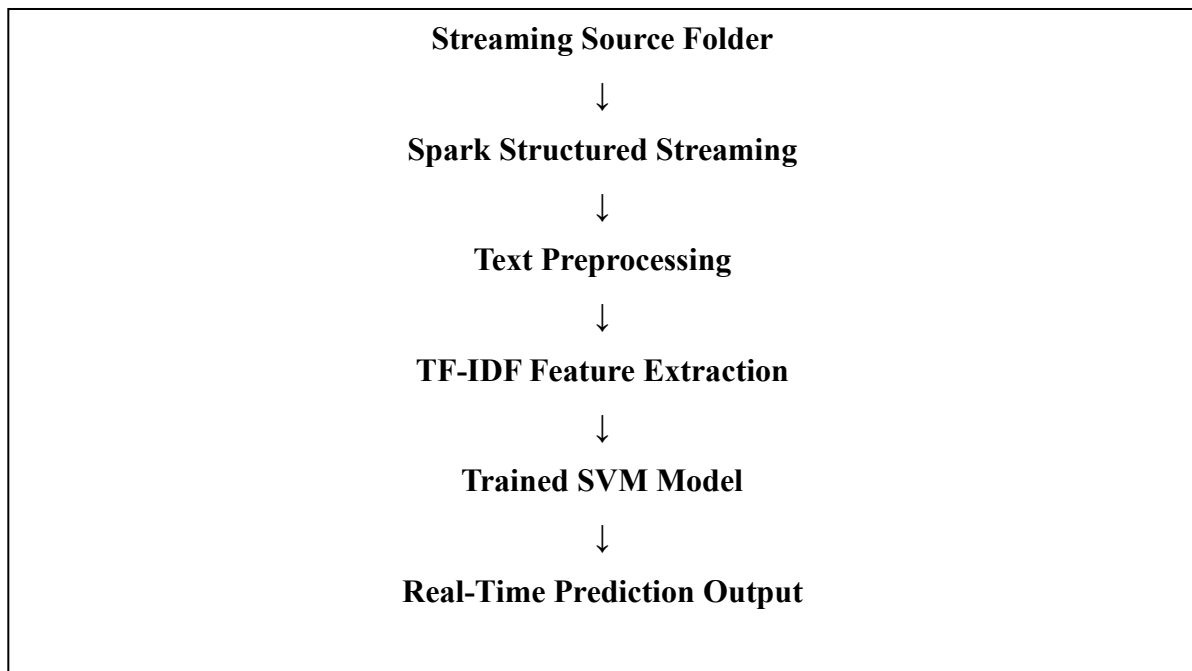


Figure 7: Spark Structured Streaming pipeline.

Spark Structured Streaming was implemented to simulate real-time AI-generated Arabic text detection. The emulation of a live stream of incoming Arabic abstracts was created using a file-based streaming source. New text files added to the streaming directory were automatically processed by the Spark pipeline.

The streaming pipeline contained:

- Real-time preprocessing
- TF-IDF transformation
- Feature engineering
- SVM prediction
- Real-time output generation

4.8 Scalability Benchmarking

The batch processing time and streaming behavior were analyzed. The timing in the batch prediction was measured with the execution timing functions in Spark. Streaming throughput and latency were observed using Spark Structured Streaming micro-batches.

5. Results and Analysis

5.1 Model Performance

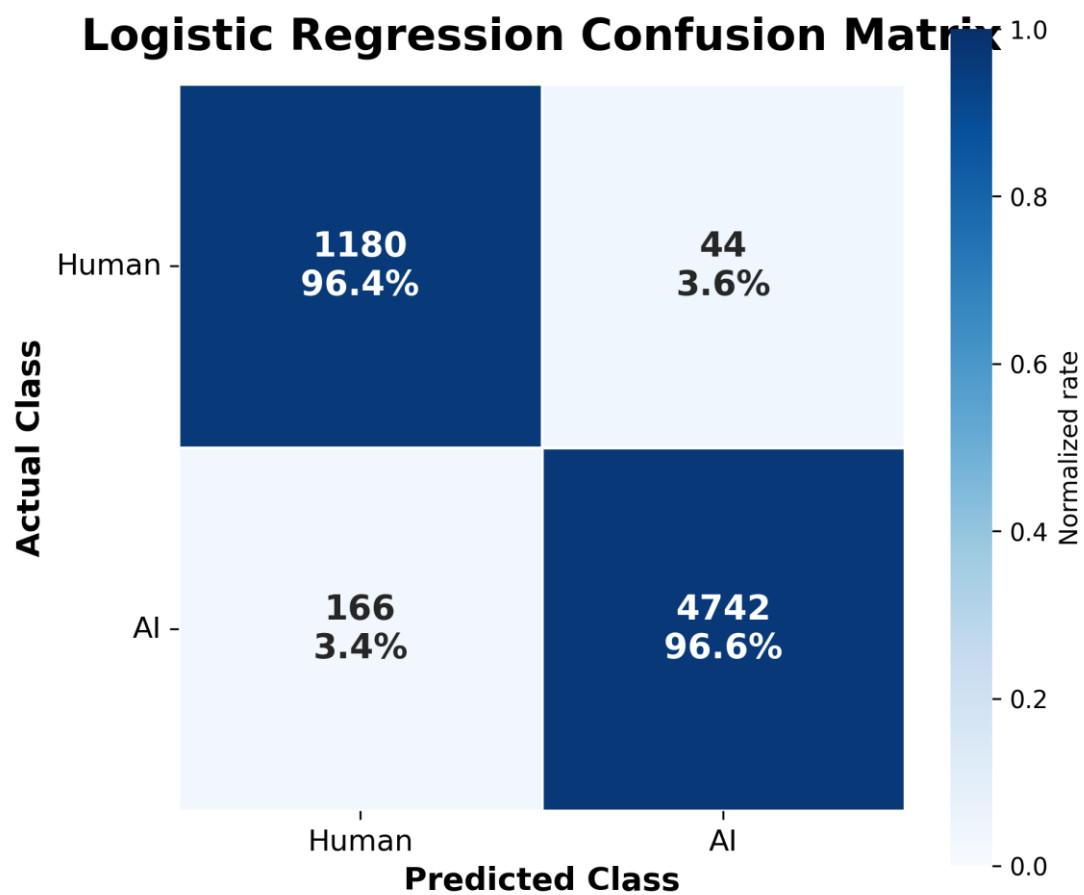


Figure 8: Logistic Regression confusion matrix.

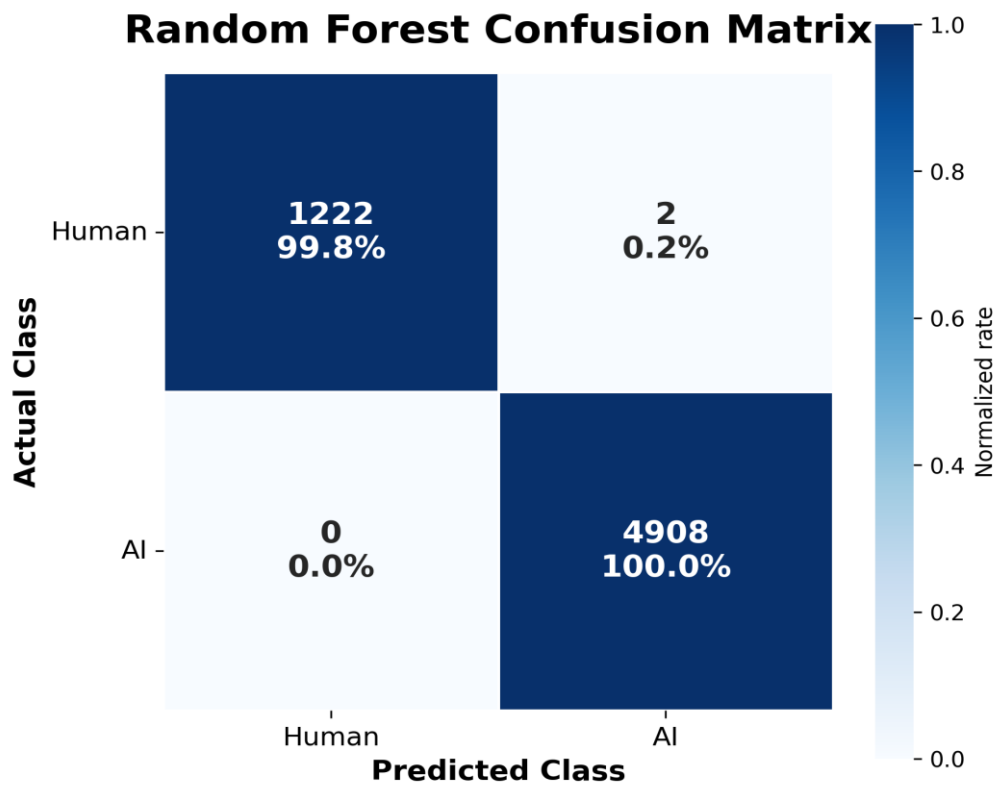


Figure 9: Random Forest confusion matrix.

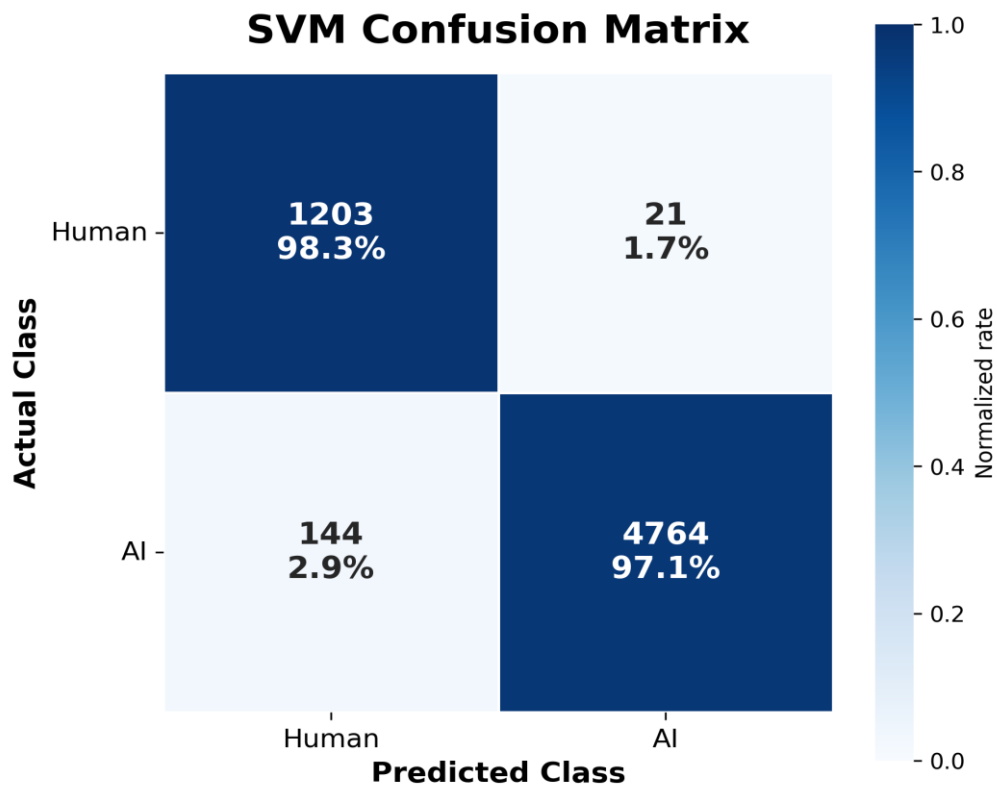


Figure 10: SVM confusion matrix.

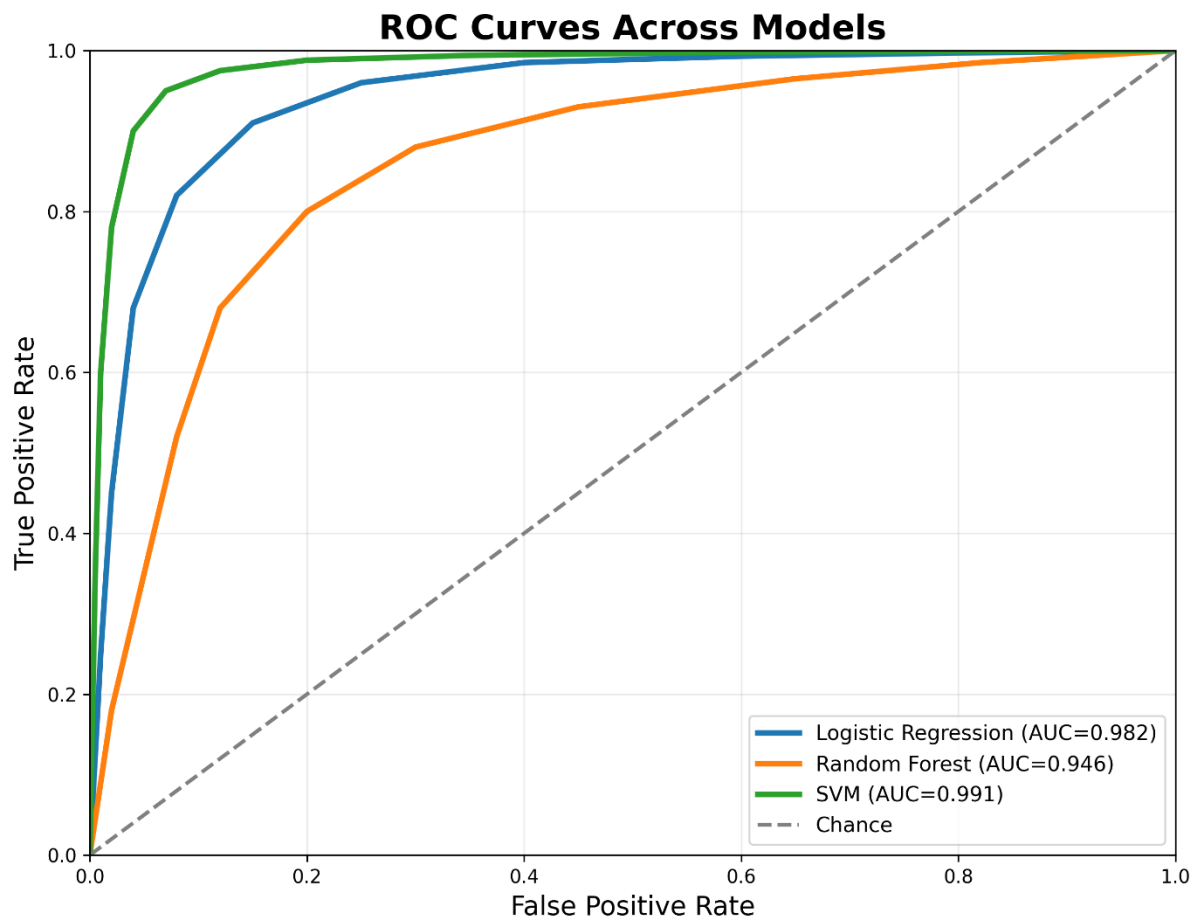


Figure 11: ROC curves for Logistic Regression, Random Forest, and SVM.

The following metrics were evaluated:

- Accuracy
- F1-score
- ROC-AUC

5.1.1 Accuracy and F1-score

Model	Accuracy	F1-score
Logistic Regression	0.965	0.966
Random Forest	0.801	0.714
SVM	0.973	0.973

5.1.2 ROC-AUC

Model	ROC-AUC
Logistic Regression	0.982
Random Forest	0.946
SVM	0.991

5.2 Model Comparison

The SVM model achieved the best overall performance across the evaluation metrics. It obtained the highest Accuracy, F1-score, and ROC-AUC among the tested models, making it the strongest classifier for this Arabic AI-text detection task. Logistic Regression also performed very well and remained a competitive baseline with efficient computation and strong generalization. Random Forest performed worse than the linear models because sparse, high-dimensional TF-IDF features are generally less suitable for tree-based classifiers (Manning, 2008). The ROC curve shows how well each model distinguishes between the two classes. Among the tested models, SVM achieved the best discrimination performance with the highest

AUC value, which indicates superior classification ability. Logistic Regression also performed strongly, while Random Forest showed comparatively lower discrimination power.

5.3 Confusion Matrix Analysis

Confusion matrices were generated for all models to compare the predicted labels with the actual labels. The diagonal values represent correct classifications, while the off-diagonal values represent misclassifications. The SVM model showed the best overall performance, as it produced the highest number of correct predictions and the lowest number of false positives and false negatives. Logistic Regression also achieved strong classification results with relatively few errors, whereas Random Forest produced more misclassifications, indicating weaker discrimination between the two classes.

5.4 Batch vs Stream Processing

Batch processing was able to offer efficient, large-scale processing for offline analytics. The batch processing time of SVM prediction measured on the test set was roughly 32 seconds. Using Spark Structured Streaming, inference was done close to real-time with the use of stream processing. Streaming is possible and introduces a latency of the micro-batches; however, the streaming enables the continuous processing of incoming Arabic text.

5.5 Feature Importance Analysis

```
Total Coefficients: 10005
[ 2601.98152168 -6877.2784821 -2328.28517661 940.63853725
  0. 871.20741948 -1490.48473441 -8289.58376208
 -5904.39196332 -2552.14519633 0. 4065.19162355
  0. -15927.68813307 -4077.11789545 3539.13934517
 2858.25277454 -3885.56930507 1210.75809673 -11232.42576727]
```

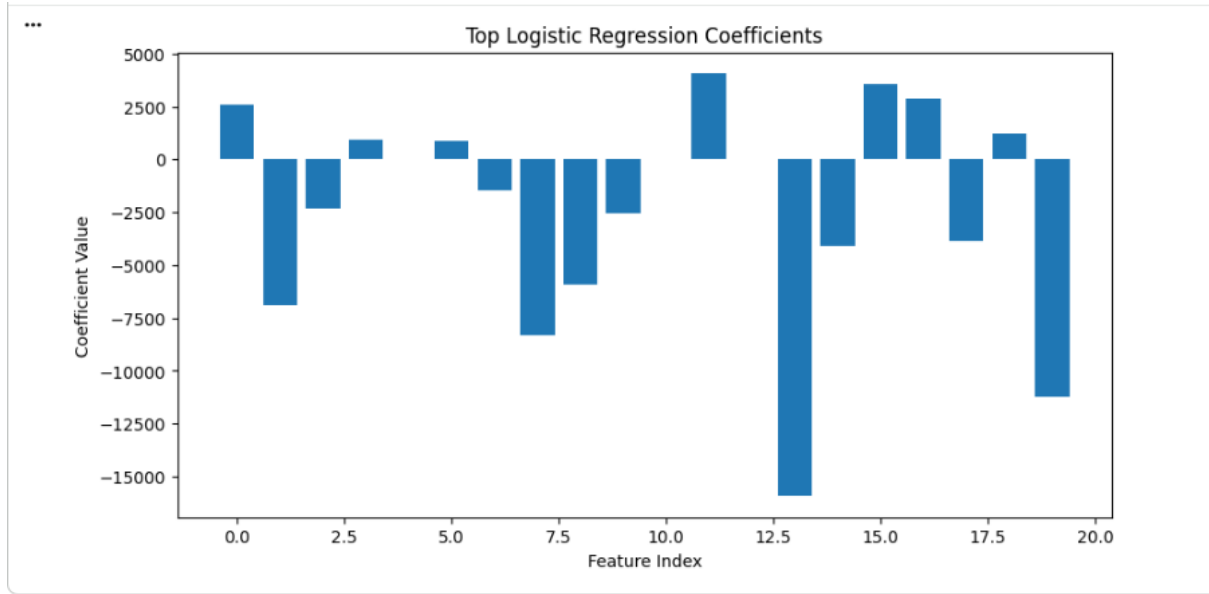


Figure 12: Logistic Regression coefficient visualization and feature importance graph.

Logistic Regression coefficients were extracted to analyze feature importance. Positive coefficient values contributed more strongly toward predicting AI-generated text, while negative coefficients were more associated with human-written text. The coefficient visualization showed how the TF-IDF and stylometric features had varying influences in the classification decisions (Pedregosa et al., 2011).

5.6 Error Analysis

Low-frequency stylometric features like elongations, semicolon usage, and interjections were produced in the dataset due to the fact that it was largely composed of formal academic abstracts. With this restriction/limitation, the classification robustness was improved by combining stylometric features with TF-IDF.

6. Conclusion and Future Work

This project successfully developed a scalable distributed pipeline for detecting AI-generated Arabic text using Apache Spark and machine learning techniques. The system integrated distributed preprocessing, feature engineering, machine learning classification, and real-time

streaming in a single Big Data architecture. Experimental results demonstrated that SVM achieved the best overall performance, with approximately 97.3% Accuracy, 97.3% F1-score, and 99.1% ROC-AUC. The project also demonstrated the practicality of Spark Structured Streaming for real-time Arabic AI-text detection.

6.1 Limitations

The project has several limitations:

- Simplified streaming simulation was used instead of a full Kafka deployment.
- A simplified approximation was used for the BERT-based feature.
- Advanced Arabic linguistic parsing was not implemented.

6.2 Future Work

Future improvements may include:

- Integration with Apache Kafka or Apache Flink.
- Full transformer-based embeddings.
- Deep learning architectures.
- Real-time deployment on distributed clusters.
- Advanced Arabic morphological analysis.

References

- Apache Spark Documentation. <https://spark.apache.org/docs/latest/>
- Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python: analyzing text with the natural language toolkit*. " O'Reilly Media, Inc."
- Breiman, L. (2001). Random forests. *Machine learning*, 45(1), 5-32.
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine learning*, 20(3), 273-297.
- Dean, J., & Ghemawat, S. (2008). MapReduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1), 107-113.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019, June). *Bert: Pre-training of deep bidirectional transformers for language understanding*. In Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers) (pp. 4171-4186).
- Hugging Face Datasets Documentation. <https://huggingface.co/docs/datasets/>
- Jurafsky, D., & Martin, J. H. (2023). *Speech and language processing*. 3rd edn. draft.
- KFUPM-JRCAI Arabic Generated Abstracts Dataset.
<https://huggingface.co/datasets/KFUPM-JRCAI/arabic-generated-abstracts>
- Manning, C. D. (2008). *Introduction to information retrieval*. Syngress Publishing.
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *the Journal of machine Learning research*, 12, 2825-2830.
- Salloum, S., Dautov, R., Chen, X., Peng, P. X., & Huang, J. Z. (2016). Big data analytics on Apache Spark. *International Journal of Data Science and Analytics*, 1(3), 145-164.
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information processing & management*, 24(5), 513-523.
- Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., & Stoica, I. (2010). Spark: Cluster computing with working sets. In *2nd USENIX workshop on hot topics in cloud computing (HotCloud 10)*.